Managing Global State

App.js

```
import React from 'react';
import './StateArena.css'; // <--- Import the CSS here
import { Provider as ReduxProvider } from 'react-redux';

import { ZustandCounter } from './zustandStore';
import { CounterProvider, ContextCounter } from './ContextStore';
import { reduxStore, ReduxCounter } from './reduxStore';
import { MobxCounter } from './mobxStore';

export default function App() {
  return (
    <div className="arena-container">
      <h1 className="arena-title">State Management Comparison</h1>

      <ReduxProvider store={reduxStore}>
        <CounterProvider>

          <div className="grid-container">
            <ZustandCounter />
            <ContextCounter />
            <ReduxCounter />
            <MobxCounter />
          </div>

        </CounterProvider>
      </ReduxProvider>

    </div>
  );
```

```
}
```

ContextStore.js

```
import React, { createContext, useState, useContext } from 'react';

// 1. CREATE: Initialize the context
const CounterContext = createContext();

// 2. PROVIDE: Create a wrapper component to hold the state
export const CounterProvider = ({ children }) => {
  const [count, setCount] = useState(0);
  const increment = () => setCount(c => c + 1);
  const decrement = () => setCount(c => c - 1);

  return (
    <CounterContext.Provider value={{ count, increment, decrement }}>
      {children}
    </CounterContext.Provider>
  );
};

// 3. CONSUME: Read the context data
export const ContextCounter = () => {
  const { count, increment, decrement } = useContext(CounterContext);
  return (
    <div className="state-card card-context">
      <h3 className="card-title">Context API</h3>
      <p className="card-count">{count}</p>
      <button onClick={increment} className="btn-action">➕</button>
      <button onClick={decrement} className="btn-action">➖</button>
    </div>
```

```
  );
};
```

mobxStore.js

```
import React from 'react';
import { makeAutoObservable } from 'mobx';
import { observer } from 'mobx-react-lite';

// 1. CREATE: Define a class and make its properties observable
class CounterStore {
  count = 0;
  constructor() {
    makeAutoObservable(this);
  }
  increment = () => { this.count += 1; }
  decrement = () => { this.count -= 1; }
}
export const mobxInstance = new CounterStore();

// 2. PROVIDE: (MobX instances can just be imported directly)

// 3. CONSUME: Wrap the component in `observer` so it tracks changes
export const MobxCounter = observer(() => {
  return (
    <div className="state-card card-mobx">
      <h3 className="card-title">MobX</h3>
      <p className="card-count">{mobxInstance.count}</p>
      <button onClick={mobxInstance.increment} className="btn-action">➕</button>
      <button onClick={mobxInstance.decrement} className="btn-action">➖</button>
    </div>
  );
```

```
});
```

reduxStore.js

```
import React from 'react';
import { configureStore, createSlice } from '@reduxjs/toolkit';
import { useSelector, useDispatch } from 'react-redux';

// 1. CREATE: Define a "Slice" (Reducer + Actions) and the Store
const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1; },
    decrement: (state) => { state.value -= 1; }
  }
});

export const { increment, decrement } = counterSlice.actions;
export const reduxStore = configureStore({
  reducer: { counter: counterSlice.reducer }
});

// 2. PROVIDE: Done in App.js using <Provider>

// 3. CONSUME: Use selectors to read, dispatch to write
export const ReduxCounter = () => {
  const count = useSelector((state) => state.counter.value);
  const dispatch = useDispatch();
  return (
    <div className="state-card card-redux">
      <h3 className="card-title">Redux Toolkit</h3>
```

```
      <p className="card-count">{count}</p>
      <button onClick={() => dispatch(increment())} className="btn-action">➕</button>
      <button onClick={() => dispatch(decrement())} className="btn-action">➖</button>
    </div>
  );
};
```

zustandStore.js

```
import React from 'react';
import { create } from 'zustand';

// 1. CREATE: Define the store and actions in one simple hook
export const useZustandStore = create((set) => ({
  count: 0,
  increment: () => set((state) => ({ count: state.count + 1 })),
  decrement: () => set((state) => ({ count: state.count - 1 })),
}));

// 2. PROVIDE: (Zustand does not require a Provider wrapper!)

// 3. CONSUME: Use the hook directly in any component
export const ZustandCounter = () => {
  const { count, increment, decrement } = useZustandStore();
  return (
    <div className="state-card card-zustand">
      <h3 className="card-title">Zustand Store</h3>
      <p className="card-count">{count}</p>
      <button onClick={increment} className="btn-action">➕</button>
      <button onClick={decrement} className="btn-action">➖</button>
    </div>
  );
```

```
};
```

StateArena.css

```
.arena-container {
  max-width: 900px;
  margin: 2rem auto;
  padding: 1rem;
  font-family: system-ui, -apple-system, sans-serif;
  text-align: center;
}

.arena-title {
  font-size: 2rem;
  font-weight: bold;
  margin-bottom: 2rem;
  color: #1e293b;
}

.grid-container {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 1.5rem;
}

/* Base Card Styling */
.state-card {
  padding: 1.5rem;
  border: 2px solid;
  border-radius: 12px;
  background-color: white;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
```

```
  transition: transform 0.2s ease;
}
.state-card:hover {
  transform: translateY(-2px);
}

.card-title {
  font-size: 1.25rem;
  font-weight: bold;
  margin-top: 0;
  margin-bottom: 1rem;
}

.card-count {
  font-size: 2.5rem;
  font-weight: bold;
  margin: 1rem 0;
  color: #0f172a;
}

/* Base Button Styling */
.btn-action {
  font-size: 1.25rem;
  padding: 0.5rem 1rem;
  margin: 0 0.5rem;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  transition: opacity 0.2s;
}
.btn-action:hover {
  opacity: 0.8;
```

```css
}

/* Zustand Theme (Blue) */
.card-zustand { border-color: #3b82f6; }
.card-zustand .card-title { color: #2563eb; }
.card-zustand .btn-action { background-color: #dbeafe; color: #1d4ed8; }

/* Context API Theme (Green) */
.card-context { border-color: #22c55e; }
.card-context .card-title { color: #16a34a; }
.card-context .btn-action { background-color: #dcfce7; color: #15803d; }

/* Redux Theme (Purple) */
.card-redux { border-color: #a855f7; }
.card-redux .card-title { color: #9333ea; }
.card-redux .btn-action { background-color: #f3e8ff; color: #7e22ce; }

/* MobX Theme (Orange) */
.card-mobx { border-color: #f97316; }
.card-mobx .card-title { color: #ea580c; }
.card-mobx .btn-action { background-color: #ffedd5; color: #c2410c; }

/* Mobile Responsiveness */
@media (max-width: 600px) {
  .grid-container {
    grid-template-columns: 1fr;
  }
}
```

This example is a comparative architectural sandbox. It demonstrates four entirely different global state management systems operating simultaneously within a single React application so students can analyze the differences in syntax, boilerplate, and data flow side-by-side.

Zustand (The Modern Minimalist): Shows how to manage global state with zero Provider wrappers and minimal code using a simple custom hook. It represents the current trend toward lightweight state managers.

Context API (The Native Approach): Demonstrates React's built-in solution. It teaches students how to create a Context.Provider wrapper to pass data down the component tree without third-party libraries, ideal for data that rarely changes (like themes or authentication).

Redux Toolkit (The Enterprise Standard): Illustrates strict, unidirectional data flow using slices, reducers, and a centralized store. It highlights the heavy boilerplate and rigid structure required for large-scale, enterprise applications.

MobX (The Object-Oriented Method): Introduces mutable, observable classes instead of React's standard immutable state architecture. It shows how wrapping a component in an observer allows it to automatically react to changes in a class property.

This architectural sandbox highlights four distinct approaches to React state management. Here is how they compare in a production environment.

State Manager	Paradigm	Boilerplate	Performance Profiling	Primary Developer Draw
Zustand	Functional / Hooks-based	Very Low	Excellent (Targeted re-renders)	Simplicity and minimal code footprint.
Context API	Native React (Component Tree)	Medium	Poor (Re-renders all consumers unless memoized)	Zero third-party dependencies.

Redux Toolkit	Unidirectional / Immutable	High	Very Good (Highly optimized selectors)	Predictability and enterprise-grade debugging.
MobX	Object-Oriented / Observable	Low	Excellent (Fine-grained reactivity)	Mutable state feels natural to OOP developers.

Architectural Breakdown

- **Zustand (The Modern Minimalist):** It abandons the traditional Provider wrapper entirely. State is stored outside the React tree in a custom hook, meaning components only re-render if the specific piece of state they subscribe to changes. It is currently the most heavily adopted alternative to Redux because it offers global state without structural lock-in.
- **Context API (The Native Approach):** Because it is built directly into React, it requires no external libraries. However, it is not a true state manager—it is a dependency injection tool. When a Context value changes, *every* component consuming that context re-renders. It is best reserved for low-frequency updates like UI themes, language localization, or user authentication status.
- **Redux Toolkit (The Enterprise Standard):** Redux forces developers into a strict, immutable, unidirectional data flow (Action → Reducer → Store → UI). While the boilerplate is heavy, this rigidity prevents "spaghetti state" in massive applications. Its time-traveling Redux DevTools make tracking complex, deeply nested data mutations highly predictable.
- **MobX (The Object-Oriented Method):** Unlike React and Redux, which demand strict immutability (creating a new copy of data to trigger a render), MobX embraces mutability. Developers mutate class properties directly (this.count++), and MobX uses a reactive graph to automatically update only the specific UI elements relying on that exact property. It bridges the gap for developers transitioning from Angular or Python/Java backgrounds.

In programming, **boilerplate** refers to repetitive, mandatory setup code that you must write just to make a tool or library work, before you can actually write the unique logic of your application. It is the "plumbing" or administrative code required by a framework's architecture.

In the context of the state management example, boilerplate is the difference between how much code you have to write to achieve the exact same result (incrementing a number).

Here is a direct comparison from your sandbox:

- **High Boilerplate (Redux Toolkit):** To increment a counter, you were forced to write a lot of architectural setup. You had to define a slice, create a reducer function, export the action creators, configure a central store, wrap your entire App.js in a <Provider store="{store}"> tag, and finally use useDispatch() and useSelector() in the component. All of this is boilerplate.
- **Low Boilerplate (Zustand):** To achieve the exact same counter, you only wrote one single useZustandStore hook containing the state and the math, and then imported it directly into the component. There were no providers, wrappers, or reducers required.